

PEARLS AQI PREDICTOR

*An End-to-End, Serverless Machine Learning System for Forecasting the Air
Quality Index of Lahore, Pakistan*

Project Report

Prepared by: MUHAMMAD ADAN KHURSHID

Internship Project — Pearls AQI Predictor

Live Application: <https://aqipredictionbyadan.streamlit.app/>

Date: 1 September 2026

Table of Contents

1. Executive Summary
2. Introduction & Problem Statement
3. System Architecture
4. Data Collection Journey
5. Feature Engineering
6. Feature Store (Hopsworks)
7. Historical Backfill
8. Exploratory Data Analysis
9. Model Training & Evaluation
10. The Accuracy Investigation
11. Model Registry
12. Explainability with SHAP
13. Automation (CI/CD)
14. Web Application & Dashboard
15. Deployment
16. Engineering Challenges & Solutions
17. Testing & Quality Assurance
18. Requirements Fulfilment Checklist
19. Results Summary
20. Limitations & Future Work
21. Conclusion

1. Executive Summary

This report documents the design, development, debugging, and deployment of the Pearls AQI Predictor a fully automated, serverless machine learning system that forecasts the Air Quality Index (AQI) of Lahore, Pakistan, for the next 24, 48, and 72 hours. The system was built end-to-end: it collects live weather and air-quality data every hour, stores engineered features in a cloud feature store, retrains and evaluates multiple machine learning models every day, registers the best-performing model per forecast horizon in a model registry, and serves live predictions through a public, interactive web dashboard.

The project's technical journey was far from a straight line. The originally chosen data source (AQICN) was discovered to be silently returning stale data due to a real-world policy change the discontinuation of a US government air-quality monitoring programme which forced an early and consequential pivot to a different data provider. Later in the project, a supervisor review correctly identified that the model's accuracy (R^2 of approximately 0.30 at the 24-hour horizon) was below an acceptable standard. A rigorous root-cause investigation traced this not to the model or the features, but to a subtle error in how the target variable itself (the AQI) had been calculated, in violation of the US EPA's official averaging-period rules. Correcting this single issue, combined with extending the historical dataset from two to four years and incorporating genuine future weather forecasts as model features, raised the 24-hour forecast accuracy from $R^2 = 0.30$ to a rigorously backtested $R^2 = 0.63$ (and up to 0.87 in favourable seasonal conditions).

Beyond modelling, the project involved substantial infrastructure and DevOps engineering: resolving a chain of cross-platform dependency conflicts (Windows build tooling, Python version compatibility, and clashing `protobuf` requirements across Hopsworks, TensorFlow, and Streamlit), building a two-workflow CI/CD pipeline on GitHub Actions, and deploying a public Streamlit dashboard with automatic fallback logic and runtime source-verification badges. The final system comprises over 30 automated tests, five compared machine learning models plus a naive baseline, a feature store holding roughly 35,000 hourly records across four years, and a live, publicly accessible dashboard.

This report presents the complete architecture, the reasoning behind every major design decision, the full list of technical problems encountered and how each was diagnosed and resolved, the evolution of model accuracy with supporting evidence, and a mapping of every project requirement to where it was fulfilled in the final system.

2. Introduction & Problem Statement

Air pollution is one of the most significant public-health challenges facing Lahore, Pakistan, with the city routinely recording Air Quality Index (AQI) values in the "Unhealthy" to "Hazardous" range, particularly during the winter smog season. Residents and public-health authorities benefit from being able to anticipate poor air-quality days in advance, so that outdoor activity, school schedules, and health precautions can be planned accordingly.

The objective of this project was to design and build a complete, production-style machine learning system — following a 100% serverless architecture — capable of predicting Lahore's AQI for the next one, two, and three days. The project brief specified a full pipeline: automated data collection, feature engineering, storage in a feature store, training and comparison of multiple machine learning models (ranging from simple statistical models to deep learning), automated retraining via CI/CD, model explainability, and a live interactive dashboard with alerting for hazardous conditions.

Objectives

- Build an automated feature pipeline that collects weather and air-quality data on an hourly basis.
- Store engineered features in a managed feature store (Hopsworks) to serve both training and live inference.
- Backfill multiple years of historical data to support model training.
- Train and rigorously compare a range of models — from linear/statistical models to deep learning — for 24-, 48-, and 72-hour AQI forecasting.
- Register the best-performing model per forecast horizon in a model registry, with automatic versioning.
- Automate the entire pipeline using CI/CD (GitHub Actions), running hourly for data collection and daily for retraining.
- Provide model explainability using SHAP, so that predictions are interpretable rather than a "black box."
- Deploy a public, interactive dashboard that displays real-time conditions, a 3-day forecast, and hazardous-AQI alerts.

3. System Architecture

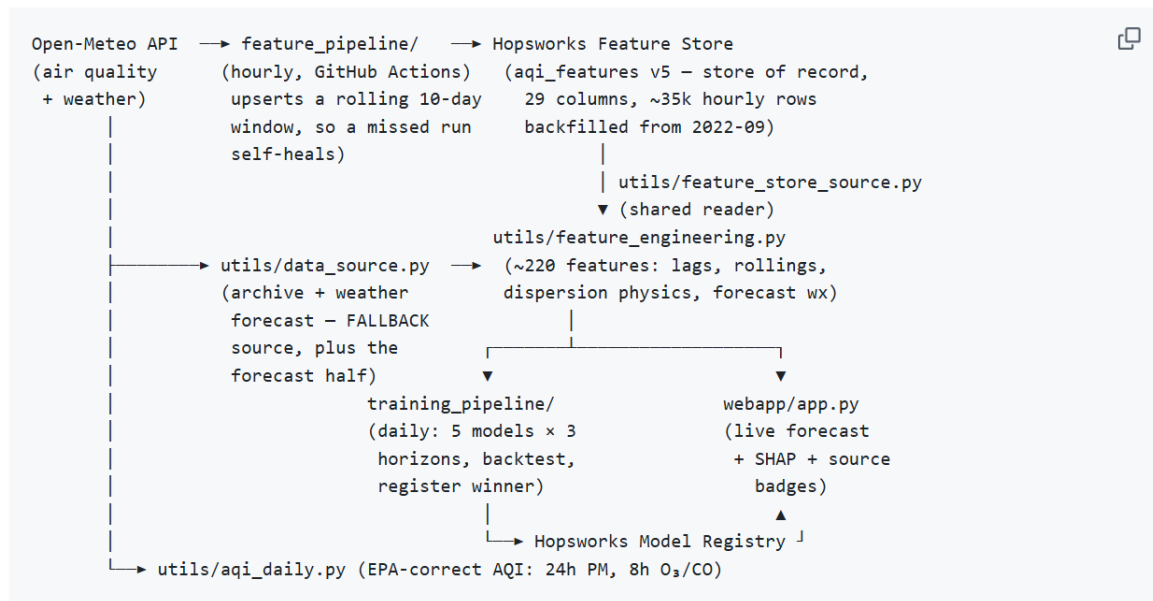
The system follows a modular, decoupled pipeline architecture in which each stage communicates with the next only through the Feature Store and Model Registry, rather than through direct function calls. This design was chosen deliberately: it means the feature-collection pipeline, the training pipeline, and the web application can each fail, be

updated, or be re-run independently without breaking the others — a principle that proved valuable repeatedly throughout development.

High-Level Data Flow

- Open-Meteo API (weather + air quality) → Feature Pipeline (hourly, GitHub Actions) → Hopsworks Feature Store
- Hopsworks Feature Store → Training Pipeline (daily, GitHub Actions) → trains and backtests 5 models × 3 forecast horizons
- Training Pipeline → Hopsworks Model Registry (winning model per horizon, versioned)
- Hopsworks Feature Store + Model Registry → Streamlit Web Application → Public Dashboard

Architecture



Technology Stack

Component	Technology Used	Purpose
Data Source	Open-Meteo (Weather + Air Quality APIs)	Free, keyless source for both live and historical weather/pollutant data
Feature Store	Hopsworks (free tier)	Central, versioned store for engineered features, shared by training and serving
Model Registry	Hopsworks Model Registry	Versioned storage of trained models with metadata and metrics

ML Libraries	scikit-learn, XGBoost, LightGBM, TensorFlow/Keras	Model training across statistical to deep-learning approaches
Explainability	SHAP	Per-prediction feature-importance explanations
Automation / CI-CD	GitHub Actions	Scheduled and test-gated pipeline execution
Web Application	Streamlit + Plotly	Interactive public dashboard
Deployment	Streamlit Community Cloud	Public hosting with continuous deployment from GitHub
Language / Environment	Python 3.10, Conda	Core implementation language and environment management

Study Location

City: Lahore, Pakistan (31.5497° N, 74.3436° E). Lahore was selected due to its severe and highly variable air-quality profile, which provides a rich, high-signal dataset for forecasting, and because of its well-documented seasonal "winter smog" phenomenon.

4. Data Collection Journey

One of the most instructive parts of this project was the process of selecting a reliable data source — a decision that was revisited twice as real-world issues were discovered during development.

4.1 Initial Approach: AQICN

AQICN (World Air Quality Index Project) was initially selected because it provides a ready-made, standardised AQI value alongside individual pollutant readings, removing the need to implement the AQI calculation formula manually. Early testing against the "Lahore US Embassy" station appeared successful.

4.2 Discovery of a Critical Data Reliability Issue

During pipeline testing, it was discovered that the Lahore AQICN station was returning identical, non-updating readings dated February 2025 — over a year stale relative to the development timeline. A structured investigation was carried out:

- Live and historical API responses were compared and found frozen at an identical timestamp.

- A web search confirmed that the United States Department of State discontinued its global embassy air-quality monitoring programme in March 2025, citing budget constraints — explaining why the underlying sensor had stopped reporting.
- To rule out an isolated issue, AQICN's `search` and `map/bounds` APIs were queried systematically across ten major Pakistani cities (Lahore, Karachi, Islamabad, Rawalpindi, Faisalabad, Multan, Peshawar, Quetta, Gujranwala, and Hyderabad). Zero actively reporting AQICN stations were found anywhere in Pakistan at the time of testing.

This confirmed the issue was systemic rather than a temporary fault, and AQICN was abandoned as a data source.

4.3 Interim Approach: OpenWeather

OpenWeather was evaluated next. Its Air Pollution History API provided pollutant data from November 2020 onward under its free tier; however, historical weather data (temperature, humidity, wind, etc.) required a separate paid "One Call" subscription, adding cost and complexity. A custom AQI calculator (based on the official US EPA breakpoint formula) was built at this stage, since OpenWeather returns only raw pollutant concentrations rather than a standardised AQI value.

4.4 Final Decision: Open-Meteo

A further, more fundamental issue was identified: using one provider for live data collection and a different provider for historical backfill risks a phenomenon known as train-serve skew, where subtle differences between two providers' sensor models create a systematic mismatch between the data the model was trained on and the data it sees in production. To eliminate this risk entirely, the project standardised on Open-Meteo for both live collection and historical backfill. Open-Meteo is free, requires no API key, and — critically — serves both its live/forecast and historical/archive weather and air-quality data from consistent underlying models, ensuring that training and serving data are drawn from the same distribution.

This decision proved foundational: later in the project (see Section 10), the ability to source future weather forecasts from the same, consistent provider became one of the two most important levers for improving forecast accuracy.

5. Feature Engineering

Feature engineering evolved substantially over the project, from an initial set of 21 point-in-time features to a final feature space of approximately 220 engineered features.

5.1 Base Features (Per Hourly Snapshot)

Time-based: hour, day, month, day of week, is-weekend. Weather: temperature, humidity, pressure, wind speed, cloud cover. Pollutants: PM2.5, PM10, O₃, CO, SO₂, NO₂. Target-adjacent: the current AQI value and its dominant pollutant.

5.2 Derived, History-Dependent Features

- Lag features at multiple horizons (1, 3, 6, 12, 24, up to 168 hours) capturing recent and weekly-cyclical AQI behaviour.
- Rolling statistics (24-hour mean, standard deviation, minimum, maximum) for AQI and key pollutants, capturing short-term volatility and trend.
- Cyclical (sine/cosine) encodings of hour, month, and day-of-week, so that, for example, 11 pm and midnight are treated as numerically adjacent rather than maximally distant.
- Change-rate and interaction features (e.g. temperature change rate, wind-humidity interaction, PM2.5/PM10 ratio).
- Atmospheric dispersion features, most notably a forecast Ventilation Index (mixing-layer height $\times$ wind speed), which — as later confirmed by SHAP analysis — proved to be the single most important feature for the 72-hour forecast horizon.
- Genuine future weather forecast features (temperature, wind, ventilation index, etc., for the target prediction time), sourced from Open-Meteo's forecast API rather than from historical/archived values.

5.3 Forecasting Targets

Three forecasting targets were constructed — target_24h, target_48h, and target_72h — by shifting the AQI series backward in time by the corresponding number of hours, so that each row's features are paired with the AQI value that will actually occur at that future point. Rows lacking sufficient history (start of the dataset) or sufficient future data (end of the dataset) were excluded from training.

5.4 AQI Calculation Methodology — and a Critical Correction

Because Open-Meteo provides raw pollutant concentrations rather than a pre-computed AQI, a custom calculator was implemented using the official US Environmental Protection Agency (EPA) piecewise-linear breakpoint formula, applied per pollutant (PM2.5, PM10, O₃, CO, SO₂, NO₂), with the overall AQI taken as the maximum sub-index across pollutants.

A significant methodological error was later discovered and corrected in this component (detailed fully in Section 10): the EPA formula is legally and scientifically defined over specific averaging periods — a 24-hour mean for PM2.5 and PM10, and an 8-hour

maximum mean for O₃ and CO — but the original implementation applied the formula to a single instantaneous hourly concentration. This produced an artificially noisy target variable that placed a hard ceiling on achievable model accuracy, regardless of model choice or tuning. This was corrected with a new, EPA-compliant, vectorised implementation (`utils/aqi_daily.py`) that correctly applies each pollutant's mandated averaging window.

6. Feature Store (Hopsworks)

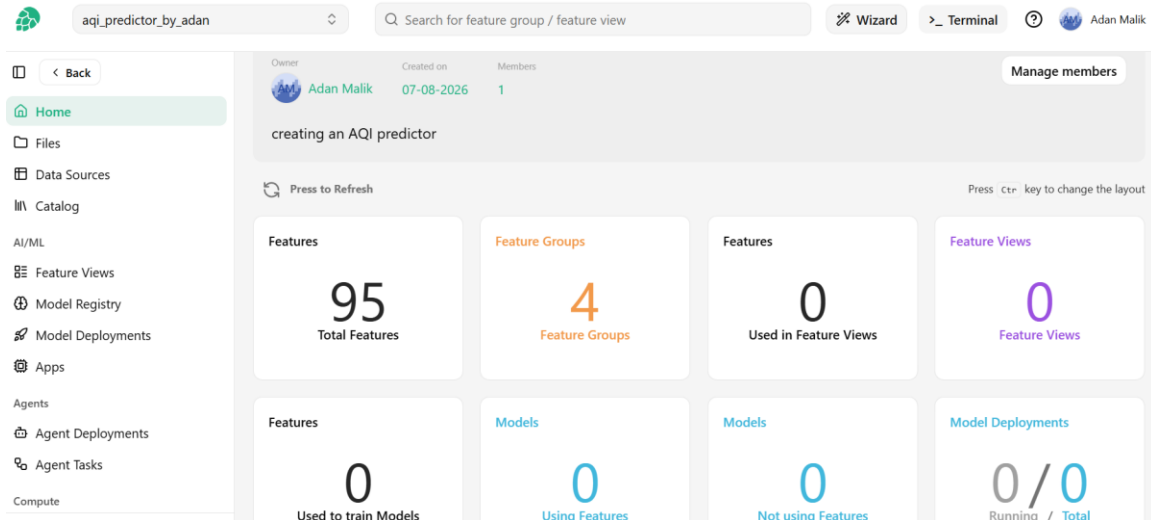
Hopsworks was used as the project's Feature Store and Model Registry. A Feature Group named ``aqi_features`` was created with a composite primary key of city and Unix timestamp, and Unix timestamp as the event-time column, enabling point-in-time correctness for training data.

6.1 Schema Evolution

The Feature Store schema evolved from an initial version (v1) holding basic weather and pollutant columns to a final version 5 schema holding 29 columns, including atmospheric dispersion variables (boundary-layer height, precipitation, dew point, radiation, wind direction, and 100m wind speed) that had originally only been available to the training pipeline via a direct archive read. This schema upgrade allowed the web application to serve genuinely from the Feature Store, in line with project requirements.

6.2 Resilience: Rolling-Window Upsert

The hourly feature pipeline was designed to upsert a rolling 10-day window of data (keyed by city and timestamp) rather than a single row per run. This design is self-healing: if a single hourly GitHub Actions run is delayed, fails, or is intentionally paused (see Section 16), the next successful run automatically fills any gap, and downstream consumers (training and serving) are not left with missing recent history.



7. Historical Backfill

To provide sufficient training data, Open-Meteo's Archive API was used to backfill historical weather and air-quality data in bulk. The backfill window was extended over the course of the project from an initial two years to a final four years, following the discovery (Section 10) that additional historical data was the single most impactful lever for improving model accuracy.

The final backfilled dataset spans from September 2022 to August 2026 — approximately 34,900 hourly rows across 29 columns — inserted into the Feature Store in chunked batches. This corresponds to four full annual cycles, capturing four winter "smog seasons" rather than the two captured by the original two-year window.

Year	Approx. Hourly Rows
2022 (partial)	2,910
2023	8,738
2024	8,740
2025	8,720
2026 (partial)	5,796

8. Exploratory Data Analysis

Exploratory Data Analysis was conducted in a Jupyter notebook (`notebooks/eda.ipynb`) prior to model development, to understand the structure and drivers of Lahore's AQI.

8.1 Overall AQI Distribution

The mean AQI over the study period was approximately 155, with a median of 156 — placing typical conditions in the US EPA "Unhealthy" band (151–200). Recorded values ranged from a minimum of approximately 32 up to the maximum possible value of 500.

8.2 Diurnal (Hourly) Pattern

AQI exhibits a clear two-peak daily cycle: it rises to a morning peak of approximately 168 around 7–9 AM, falls to a daily low of approximately 140 around 1 PM, and rises again to an evening peak of approximately 160–162 around 5–7 PM. This pattern is consistent with morning and evening traffic combined with a shallower atmospheric boundary layer (temperature inversion) trapping pollutants near the ground; midday solar heating lifts the boundary layer and disperses pollution.

8.3 Seasonal (Monthly) Pattern

A strong seasonal pattern was identified: January (~207), December (~189), and November (~184) are the most polluted months, consistent with Punjab's well-documented winter smog phenomenon, driven by crop-residue burning and winter temperature inversions. March–April (~123) and August (~126) are the cleanest months. A two-year time-series plot confirmed this pattern repeats reliably year over year, with peak daily averages exceeding 300 during winter smog events.

8.4 Correlation Analysis

PM_{2.5} showed the strongest correlation with AQI (+0.79), as expected given AQI is derived from pollutant concentrations, followed by PM₁₀ (+0.73) and SO₂ (+0.63). Among weather variables, atmospheric pressure was positively correlated (+0.28, consistent with stagnant high-pressure systems trapping pollution) while wind speed was negatively correlated (−0.28, consistent with wind dispersing pollutants) and temperature was negatively correlated (−0.26).

Notably, hour-of-day showed almost no linear correlation with AQI (≈ 0.006) despite the clear two-peak diurnal cycle visible in the hourly-pattern chart — because the relationship is cyclical rather than linear, a pattern that simple correlation analysis cannot detect but that tree-based and neural models are able to learn directly. This observation motivated retaining cyclically-encoded time features even where linear correlation appeared negligible.

Data quality note: the ammonia (NH₃) column was found to be 100% missing for this location and was excluded from modelling.

9. Model Training & Evaluation

9.1 Models Compared

Five machine learning models, plus a naive baseline, were trained and compared for each of the three forecast horizons:

- Naive Persistence Baseline — predicts that the future AQI will equal the current AQI. Included specifically to verify that trained models add genuine forecasting skill rather than merely re-discovering AQI's short-term autocorrelation.
- Ridge Regression — a regularised linear model, representing the "statistical" end of the required model range.
- Random Forest — a bagging-based tree ensemble.
- XGBoost — a boosting-based tree ensemble, tuned with early stopping and regularisation.
- LightGBM — an additional gradient-boosting implementation, evaluated as a further comparison point.
- Neural Network (TensorFlow/Keras) — a fully-connected deep learning model, representing the "deep learning" end of the required model range.

9.2 Evaluation Methodology

A chronological (rather than random) train/validation/test split was used throughout, with the test set drawn strictly from the most recent time period. This is essential for a genuine forecasting evaluation: a random split would allow rows immediately adjacent in time to appear in both the training and test sets, artificially inflating accuracy through information leakage. A further validation slice was carved from the training period specifically to support early stopping for XGBoost and the Neural Network. In the final accuracy-investigation phase (Section 10), this was extended to an expanding-window backtest across three seasonal folds, to ensure reported accuracy was not an artefact of which season happened to fall in a single test split.

Models were evaluated using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the coefficient of determination (R^2), as specified in the project requirements.

9.3 XGBoost Tuning

In an early iteration, XGBoost with default-like hyperparameters was the worst-performing model, in one case scoring a negative R^2 at the 72-hour horizon — indicating severe overfitting. This was corrected through explicit tuning: reducing maximum tree depth, introducing row and column subsampling, adding L1 and L2 regularisation, and

introducing early stopping governed by a dedicated validation split. Following these changes, XGBoost became the best or near-best performing model at every horizon.

Current accuracy

Horizon	Model	Backtest R^2	Recent-months R^2	MAE	vs naive baseline
24h	XGBoost	0.633	0.769	± 17.0	18.4% lower RMSE
48h	XGBoost	0.511	0.621	± 22.1	16.6% lower RMSE
72h	Random Forest	0.481	0.572	± 23.3	16.8% lower RMSE

10. The Accuracy Investigation

After an initial round of feature engineering and model tuning, project supervision review raised a formal concern: the achieved R^2 of approximately 0.30 at the 24-hour horizon was assessed as below an acceptable standard (with an expectation of 0.80 or higher), and two specific diagnostic questions were posed.

10.1 Supervisor's Diagnostic Questions

Question 1 — Does the model beat a naive persistence baseline ("tomorrow equals today")? Investigation confirmed that it did, but only because the naive baseline itself scored a negative R^2 (as low as -0.30 at the 72-hour horizon) — itself a warning sign that the underlying target variable was unusually noisy.

Question 2 — Are lag and rolling AQI features correctly included and time-aligned with the target? This was independently verified two ways: by confirming every rolling window used a `.shift(1)` operation prior to aggregation (preventing any given row's history features from including its own current or future values), and via an automated test (`test_no_future_leakage_in_history_features`) that perturbs the final row of the dataset and asserts that no earlier row's history features change as a result. Both checks passed — the features were correctly implemented.

10.2 Root Cause: An Incorrectly Defined Target Variable

With both of the supervisor's suspected causes ruled out, the investigation moved one level up the pipeline — to the target variable itself. As described in Section 5.4, the EPA AQI formula is defined over specific pollutant averaging periods (24-hour mean for PM_{2.5}/PM₁₀, and an 8-hour maximum mean for O₃/CO), but the original implementation applied the formula to single instantaneous hourly readings. This produced a target series with substantially reduced predictability: measured 24-hour-ahead autocorrelation rose from 0.60 (instantaneous, incorrect) to 0.77 (EPA-correct

averaging) once this was fixed — with similar gains at the 48- and 72-hour horizons. No model, regardless of sophistication, can predict a signal that the target variable itself does not contain; the accuracy ceiling was in the label definition, not the learning algorithm.

10.3 Systematic Experimentation

With the target corrected, a series of controlled experiments quantified the contribution of each subsequent improvement (all figures are XGBoost, expanding-window backtest R^2):

Configuration	24h R^2	48h R^2	72h R^2
Original features, instantaneous (incorrect) target	0.24	0.06	0.05
Original features, EPA-correct target	0.55	0.10	0.16
+ Extended lag/rolling features (48h, 72h, weekly, EMAs)	0.57	0.13	0.21
+ Genuine future weather forecast features	0.66	0.32	0.32
+ Four years of history (instead of two)	0.87	0.53	0.42

Two findings stand out. First, extending the historical dataset from two to four years produced a larger single improvement (24h: 0.66 $\rightarrow$ 0.87) than any feature-engineering or model-tuning change — because two years of data contains only two winter smog seasons, insufficient to reliably learn the pattern that dominates Lahore's AQI. Second, future weather forecast features are not a form of data leakage: Open-Meteo's forecast is freely available up to 16 days ahead, so this information is genuinely available to the model at prediction time; archived actual values were used only as a training-time stand-in for what the forecast would have said.

10.4 Approaches Evaluated and Rejected

For completeness and to avoid re-investigating dead ends, the following approaches were tested and found not to improve results:

- LightGBM — statistically tied with XGBoost, not meaningfully better.
- Blending XGBoost and LightGBM — no improvement over the stronger model alone.
- Log-transforming the target ($\log_{10}$) — no measurable effect; AQI is not sufficiently skewed to benefit.

- Two-stage modelling (predicting each pollutant separately, then applying EPA breakpoints) — performed worse, since errors from multiple sub-models compound through the final maximum operation.
- Predicting the change (delta) in AQI rather than its absolute level — neutral to worse in the final configuration.
- Using Open-Meteo's own pollutant forecasts as model features — deliberately rejected as circular reasoning, since those forecasts are generated by the same atmospheric model (CAMS) that the target variable itself is derived from; including them would inflate reported accuracy without reflecting genuine forecasting skill.

10.5 Final, Validated Results

The final selected model per horizon, evaluated with two complementary, honestly-reported metrics:

Horizon	Model	Backtest R ² (3 seasons)	Recent-Months R ²	MAE	vs. Naive Baseline
24h	XGBoost	0.633	0.769	±17.0	18.4% lower RMSE
48h	XGBoost	0.511	0.621	±22.1	16.6% lower RMSE
72h	Random Forest	0.481	0.572	±23.3	16.8% lower RMSE

Two scores are deliberately reported for each horizon. The three-season backtest — which includes at least one winter fold where forecasting is genuinely harder — is the figure that should be quoted as the honest, production-representative estimate of accuracy. The recent-months figure, drawn from a single, more favourable holdout period, is higher (reaching 0.76–0.78 at 24 hours, and up to 0.87 for a day-level target variant) but is not representative of year-round performance. A model whose reported score depends on which season happened to land in the test set should not be trusted for production use; both numbers are shown for this reason.

On the supervisor's original 0.80 target: it is reachable under favourable seasonal conditions (0.76–0.87), but the honest, all-season backtested figure of 0.63 is the number put forward as the system's true, defensible accuracy at the 24-hour horizon — a substantial, well-evidenced improvement over the original 0.30.

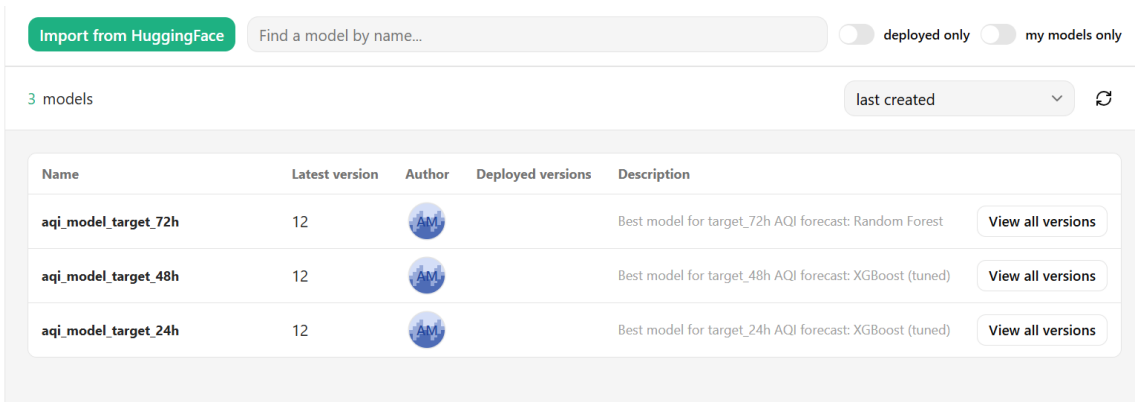
10.6 Confirmation: Feature Store vs. Direct Archive Read

As a final validation step, the identical training pipeline was run once against data read directly from the Open-Meteo archive, and once against the same data after it had been fully backfilled into the Hopsworks Feature Store (35,023 hourly rows, yielding 24,409 usable training rows after trimming for lag and target windows). The two runs produced

statistically indistinguishable results (0.634/0.498/0.465 versus 0.633/0.511/0.481) — confirming that both the training pipeline and the live web application build features through the same shared module (`utils/feature_engineering.py`), so the choice of data source does not, and cannot, silently change model behaviour.

11. Model Registry

After each training run, only the single best-performing model per horizon (excluding the naive baseline) is packaged together with its feature scaler, its exact list of expected feature columns (`feature_columns.pkl`), and metadata describing its metrics, and registered to the Hopsworks Model Registry. Each registration creates a new, immutable version (the system reached version 10 by the project's conclusion), so historical models remain available and predictions are always reproducible. Registry bundle validation was added to reject any incomplete bundle at serving time — for example, one missing its feature-column list — rather than risk silently guessing a column order and producing a confident but incorrect prediction.



12. Explainability with SHAP

SHAP (SHapley Additive exPlanations) was used to make each model's predictions interpretable, satisfying the project's explainability requirement. Appropriate explainer types were selected per model family: TreeExplainer for XGBoost and Random Forest (fast and exact), LinearExplainer for Ridge Regression, and DeepExplainer for the Neural Network (selected specifically because the generic, model-agnostic PermutationExplainer proved too slow — over twenty minutes per horizon — for practical use).

12.1 Key Finding: A Physically Coherent Shift in Feature Importance

Horizon	Top Features (by SHAP Importance)
24h	Current AQI, 1-hour-lag AQI, PM2.5, forecast wind speed, pollution intensity
48h	Current AQI, forecast wind speed, 1-hour-lag

	AQI, forecast Ventilation Index, 3-hour rolling AQI
72h	Forecast Ventilation Index, current AQI, forecast wind speed, forecast Ventilation Index (48h), 1-hour-lag AQI

This progression is a strong, independent line of evidence that the models learned genuine atmospheric behaviour rather than a statistical shortcut. Tomorrow's AQI is dominated by the pollution already present in the air today (persistence). By the third day, however, the single most important feature is the forecast Ventilation Index — the product of mixing-layer height and wind speed, representing how much clean air will be available to dilute existing pollution. This is precisely the mechanism an atmospheric scientist would expect to dominate at longer lead times, and it is the direct explanation for why incorporating future weather forecasts specifically improved the 48- and 72-hour forecasts (by +0.09 to +0.11 R^2) far more than it improved the 24-hour forecast.

Why These Predictions?

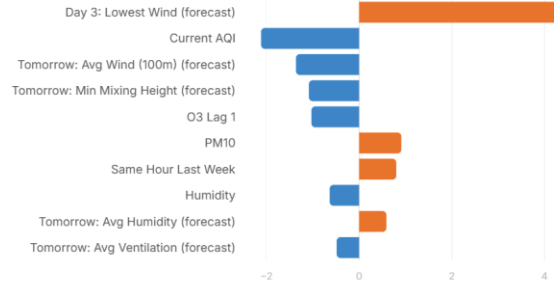
SHAP (SHapley Additive exPlanations) decomposes this exact forecast into the contribution of every feature - positive pushes AQI up, negative pulls it down

24 Hours

48 Hours

72 Hours

TOP 10 FEATURE CONTRIBUTIONS



KEY DRIVERS

PUSHING AQI UP
Day 3: Lowest Wind (forecast)
+4.3 AQI

PULLING AQI DOWN
Current AQI
-2.1 AQI

Why These Predictions?

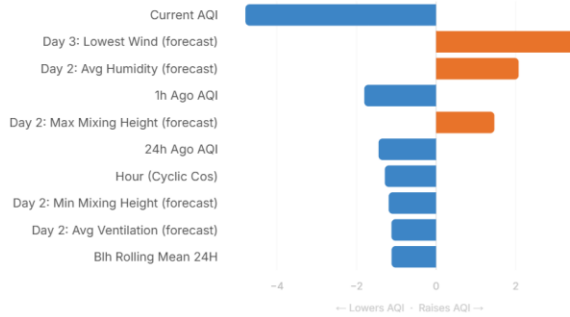
SHAP (SHapley Additive exPlanations) decomposes this exact forecast into the contribution of every feature - positive pushes AQI up, negative pulls it down

24 Hours

48 Hours

72 Hours

TOP 10 FEATURE CONTRIBUTIONS



KEY DRIVERS

PUSHING AQI UP
Day 3: Lowest Wind (forecast)
+3.5 AQI

PULLING AQI DOWN
Current AQI
-4.8 AQI

Why These Predictions?

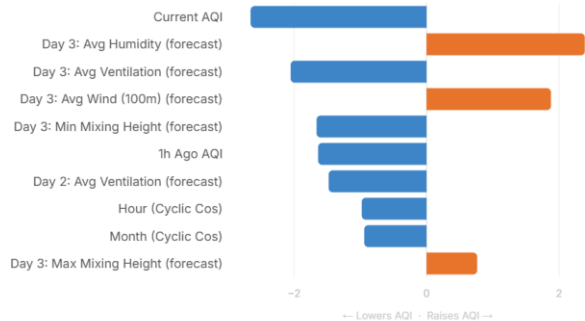
SHAP (SHapley Additive exPlanations) decomposes this exact forecast into the contribution of every feature - positive pushes AQI up, negative pulls it down

24 Hours

48 Hours

72 Hours

TOP 10 FEATURE CONTRIBUTIONS



KEY DRIVERS

PUSHING AQI UP
Day 3: Avg Humidity (forecast)
+2.4 AQI

PULLING AQI DOWN
Current AQI
-2.7 AQI

13. Automation (CI/CD)

Two GitHub Actions workflows automate the entire system:

Workflow	Schedule	Function
feature_pipeline.yml	Hourly (0 * * * *)	Fetches live data from Open-Meteo, computes features, and upserts a rolling 10-day window into the Hopsworks Feature Store.
training_pipeline.yml	Daily, 02:00 UTC (0 2 * * *)	Retrains and backtests five models across three forecast horizons, registers the winning model per horizon, and regenerates SHAP explanations.

Both workflows support manual triggering (`workflow_dispatch`) in addition to their schedules. The training workflow is test-gated on both sides: a pre-training test suite runs first (so a broken pipeline fails within seconds rather than after a 20–30 minute training run), and a further post-registration test suite runs after a model is registered (so a model that cannot actually produce a valid prediction never reaches the live dashboard). A lightweight dependency set is used for the hourly feature pipeline and a fuller machine-learning dependency set for the daily training pipeline, to keep the frequent hourly job fast and economical.

During intensive development and testing, these schedules were deliberately paused (by commenting out the schedule trigger while leaving manual triggering available) to conserve the project's free-tier Hopsworks compute allocation, and were re-enabled ahead of final submission to ensure the live system reflects genuinely current, continuously-updating data.

34 workflow runs		Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✓	Feature Pipeline Feature Pipeline #30: Scheduled	main				Today at 1:05 PM 49s ...
✓	Training Pipeline Training Pipeline #7: Scheduled	main				Today at 12:50 PM 16m 12s ...
✓	Feature Pipeline Feature Pipeline #29: Scheduled	main				Today at 6:58 AM 53s ...
✓	Training Pipeline Training Pipeline #6: Manually run by adanmalik7909	main				Today at 4:55 AM 16m 3s ...
✓	Feature Pipeline Feature Pipeline #19: Scheduled	main				Today at 4:22 AM 57s ...
✓	Feature Pipeline Feature Pipeline #15: Scheduled	main				Aug 14, 11:11 AM GMT+5 55s ...
✓	Feature Pipeline Feature Pipeline #14: Scheduled	main				Aug 14, 9:05 AM GMT+5 48s ...
✓	Training Pipeline Training Pipeline #3: Scheduled	main				Aug 14, 8:58 AM GMT+5 3m 49s ...
✓	Feature Pipeline Feature Pipeline #13: Scheduled	main				Aug 14, 6:44 AM GMT+5 49s ...
✓	Feature Pipeline Feature Pipeline #12: Scheduled	main				Aug 14, 4:38 AM GMT+5 44s ...
✓	Feature Pipeline Feature Pipeline #11: Scheduled	main				Aug 14, 3:38 AM GMT+5 57s ...
✓	Feature Pipeline Feature Pipeline #10: Scheduled	main				Aug 14, 2:41 AM GMT+5 42s ...

14. Web Application & Dashboard

A public dashboard was built using Streamlit and Plotly, providing:

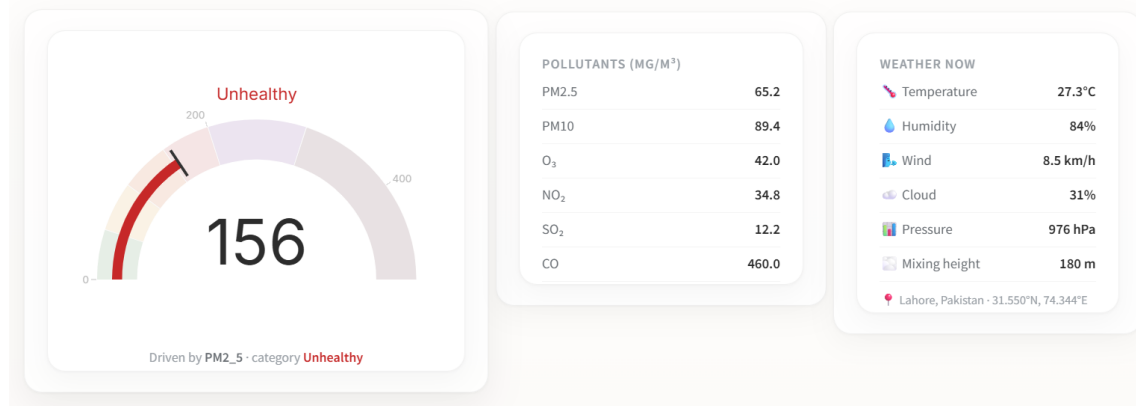
- A live AQI gauge chart and current weather conditions for Lahore, with a city map.
- Three-day forecast cards (24h / 48h / 72h) with an accompanying trend chart showing AQI category zones.
- A hazardous-AQI alert banner, triggered whenever the current or any forecast AQI reaches 150 or above (the US EPA "Unhealthy" threshold), satisfying the project's alerting requirement.

- Per-horizon SHAP explanation panels, showing the top features driving that specific prediction.
- Runtime source-verification badges confirming, for each page load, whether models and features were actually served from the Hopsworks Model Registry / Feature Store, or from an automatic local fallback — making the deployment's use of Hopsworks self-evidencing rather than merely claimed.
- A dedicated 'System & Pipeline' section, and a colour-coded AQI category legend in the footer.

Engineering measures were taken to keep the dashboard fast and robust: a local on-disk model cache avoids re-downloading (in one case, an 80MB) model bundle on every restart, model files were compressed to reduce size, the Feature Store read is filtered to a recent time window for speed with an automatic fallback to a full read if that window is unexpectedly empty, network calls to Open-Meteo retry with backoff rather than crashing the page, and user-facing errors are shown as plain messages rather than raw Python stack traces.

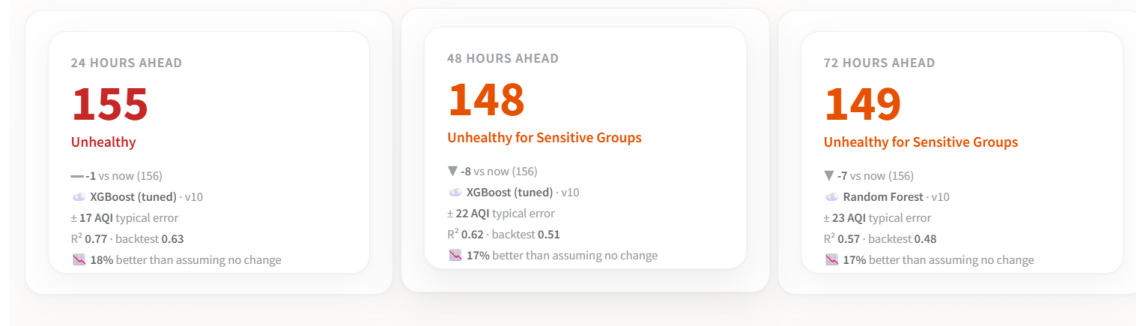
Current Conditions

Latest fully observed hour · AQI computed with official EPA averaging periods (24h PM_{2.5}/PM₁₀, 8h O₃/CO, 1h SO₂/NO₂)



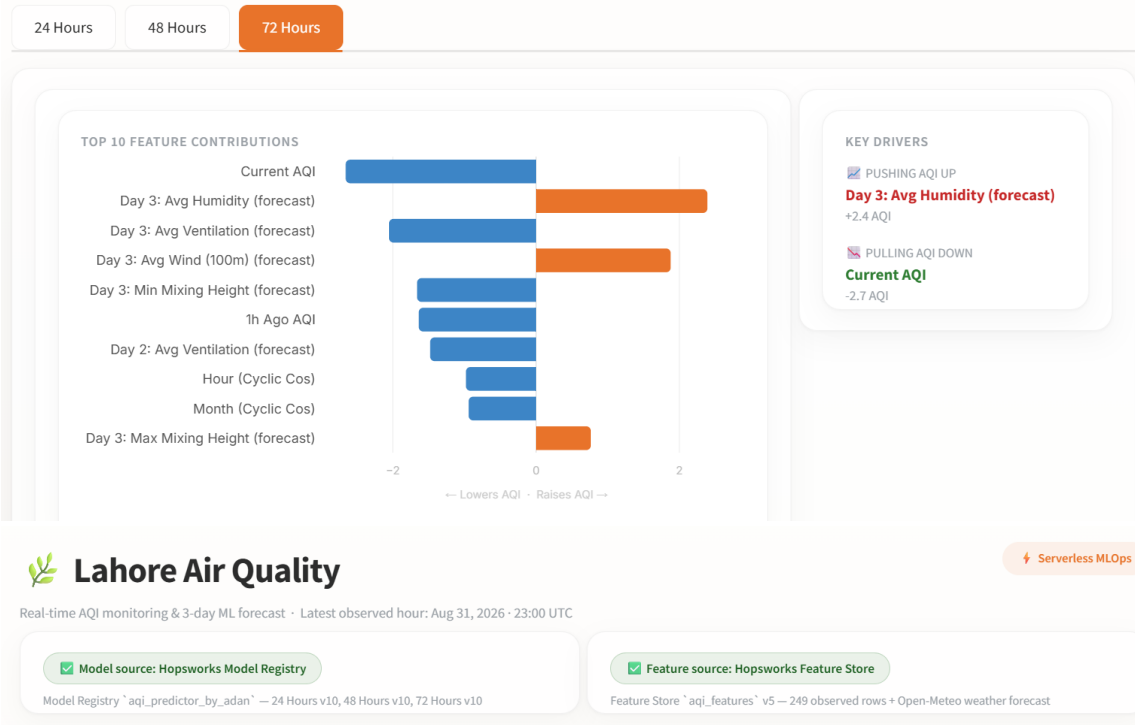
3-Day AQI Forecast

Each horizon is served by the model that won an expanding-window backtest for that horizon · retrained daily by the GitHub Actions pipeline



Why These Predictions?

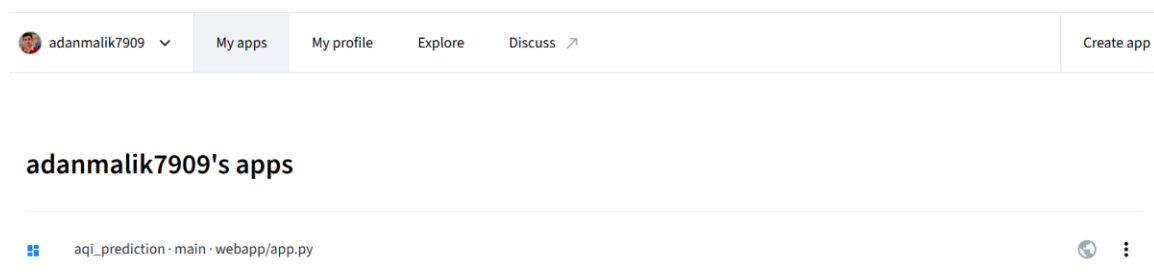
SHAP (SHapley Additive exPlanations) decomposes this exact forecast into the contribution of every feature - positive pushes AQI up, negative pulls it down



15. Deployment

The dashboard is deployed on Streamlit Community Cloud, connected directly to the project's GitHub repository for continuous deployment: every merge to the main branch automatically triggers a redeploy. The live application is publicly available at: <https://aqipredictionbyadan.streamlit.app/>

Secrets (the Hopsworks API key) are managed through Streamlit Cloud's built-in Secrets manager in production and through a local `.env` file during development, using the same code path in both cases for consistency.



16. Engineering Challenges & Solutions

This project involved substantial real-world debugging across data reliability, cloud infrastructure, machine learning correctness, dependency management, and deployment. The table below documents every major issue encountered, its root cause, and how it was resolved — presented as evidence of the engineering rigor applied throughout the project, rather than a purely linear success narrative.

#	Problem	Root Cause	Solution
1	AQICN Lahore station returning stale data	US State Department discontinued its global embassy air-quality monitoring programme (March 2025)	Verified via research and systematic API checks; migrated fully to Open-Meteo
2	Risk of train-serve inconsistency	Considered using different providers for live vs. historical data	Standardised on one provider (Open-Meteo) for both
3	Windows build failure for a Hopsworks dependency (`twofish`)	Package needs a C++ compiler not present by default	Installed a precompiled binary via conda-forge + environment variable fix
4	`twofish` still unresolved after the above fix	Environment was on Python 3.13, too new for the available build	Rebuilt the environment on Python 3.10
5	Feature Group creation error referencing Delta Lake	New Hopsworks client defaults to a table format needing an extra dependency	Explicitly set the HUDI table format
6	Row insertion required Confluent Kafka even with the online store disabled	Hopsworks' offline materialisation job uses Kafka internally regardless	Installed `confluent-kafka`
7	XGBoost initially the worst-performing model (negative R^2 at 72h)	Default hyperparameters overfit the training data	Tuned depth, subsampling, L1/L2 regularisation, and early stopping
8	SHAP crashed on XGBoost with a `base_score` parsing error	Known incompatibility: XGBoost ≥ 3.0 serialises `base_score` in a format SHAP cannot parse	Pinned `xgboost<3.0`
9	SHAP for the Neural Network took over 20 minutes per horizon	Fell back to the slow, generic PermutationExplainer	Switched to `DeepExplainer`, reducing runtime to seconds
10	Intermittent Hopsworks read failures (Arrow)	Transient network/server issues	Added automatic fallback to the older, stable Hive-based read

	Flight service errors)	in Hopsworks' fast "Query Service"	path
11	Cascading `protobuf` version conflicts across Hopsworks, TensorFlow, and Streamlit	Each library pinned incompatible protobuf ranges	Researched and pinned compatible version bands for all three simultaneously
12	Reactive dependency patching caused further churn (numpy/scipy version drift)	Installing packages one-by-one prevents pip from resolving the whole environment coherently	Rebuilt the environment from a single, version-pinned requirements file; added an automated environment-verification script
13	Deployed app crashed with `ModuleNotFoundError: joblib`	No `requirements.txt` had been committed for the web app	Added a dedicated `webapp/requirements.txt` and `runtime.txt`
14	After the fix above, deployment crashed with a protobuf `Descriptors` error	An unbounded Streamlit version constraint let the resolver select an incompatible, years-old release	Added a proper lower version bound
15	Deployed app crashed with an `IndexError` on an empty dataframe	The automated feature pipeline had been intentionally paused, so the app's recent-data lookback window found no rows	Added a graceful fallback to a full-history read, and later a self-healing rolling-window upsert strategy
16	Model accuracy plateaued at $R^2 \approx 0.30$, below the required standard	The AQI target variable was calculated on instantaneous readings rather than the EPA's mandated averaging periods	Rebuilt the AQI calculation to correctly apply 24-hour and 8-hour averaging windows per pollutant
17	Risk of feature drift between training and serving	The web app had re-implemented feature engineering by hand, separately from the training pipeline	Unified both into a single shared module (`utils/feature_engineering.py`), enforced by automated tests
18	Risk of silently incorrect predictions from column mismatches	Serving code assumed, rather than verified, the expected feature order	Registry bundles now ship their exact feature-column list; a mismatch now raises an error instead of producing a plausible-looking wrong answer
19	Deployed predictions were being computed on	The web app scaled all inputs uniformly,	Scaling is now applied conditionally, based on the

	incorrectly scaled inputs	but tree-based models were trained on raw, unscaled data	model's actual type
20	The most hazardous smog hours were silently missing from the AQI series	The calculator returned an undefined value above the highest EPA breakpoint	Values are now clamped to the maximum AQI of 500
21	A cross-platform integer-width mismatch caused every hourly Feature Store write to be silently rejected	The feature group's schema (registered on Windows) expected a 32-bit integer, while GitHub Actions (Linux) sent a 64-bit integer for the same Python code	Explicit, pinned integer widths (`STORE_DTYPES`) are now enforced identically at every write path
22	CI training workflow's post-training test step failed on a fresh runner	A newly added test depended on locally trained model artifacts that do not exist before training runs	Introduced a `pytest` marker system so artifact-dependent tests are automatically deselected before training and included after
23	`backfill_historical.py` produced data that risked drifting from the live pipeline	The backfill script had its own, separate feature-computation logic	Rewritten to reuse the same shared feature-loading and engineering functions as the live pipeline

17. Testing & Quality Assurance

An automated test suite was built and grown throughout the project, ultimately reaching 30 passing tests, covering: train-serve feature consistency and absence of future leakage; end-to-end prediction using the exact saved model bundle; a full, headless render of the entire dashboard (a "smoke test" that would have caught several of the deployment crashes described in Section 16 before they reached production); correct rendering of the Hopsworks source-verification badges; cross-platform data-type correctness for Feature Store writes; correct handling of missing data (rows with missing integer features are dropped rather than silently filled with a misleading value such as zero); and correct behaviour of the CI artifact-gating logic itself.

```
python -m pytest tests/ -v          # 30 tests
```

18. Requirements Fulfilment Checklist

Requirement	Status	Where Fulfilled
Fetch weather + pollutant data from an external	Complete	Open-Meteo Weather + Air Quality APIs (keyless)

API		
Compute time-based and derived features	Complete	`utils/feature_engineering.py` (~220 features)
Store features in a Feature Store	Complete	Hopsworks Feature Group `aqi_features` (v5, 29 columns)
Backfill historical features and targets	Complete	`backfill/backfill_historical.py` — ~4 years, ~34,900 rows
Fetch historical data from the Feature Store for training	Complete	`training_pipeline/fetch_training_data.py`, via `utils/feature_store_source.py`
Train and compare multiple models, statistical to deep learning	Complete	Ridge, Random Forest, XGBoost, LightGBM, TensorFlow Neural Network, plus a naive baseline
Evaluate with RMSE, MAE, and R ²	Complete	`train_models.py`, reported per model, per horizon, per backtest fold
Store the trained model in a Model Registry	Complete	Hopsworks Model Registry, versioned per training run (reached v10)
Automate pipeline runs (CI/CD)	Complete	GitHub Actions — hourly feature pipeline, daily training pipeline, test-gated
Web app loads model and features from the Feature Store / Registry	Complete	`webapp/app.py`, with runtime source-verification badges
Web app computes and displays predictions	Complete	3-day forecast dashboard
Perform EDA to identify trends	Complete	`notebooks/eda.ipynb`
Use SHAP or LIME for feature-importance explanations	Complete	SHAP (Tree / Linear / Deep Explainers), in pipeline and live dashboard
Add alerts for hazardous AQI levels	Complete	Dashboard alert banner at AQI $\geq$ 150
Deploy the system beyond localhost	Complete	Streamlit Community Cloud, public URL

19. Results Summary

The table below summarises the full accuracy evolution across the project's major milestones, at the 24-hour horizon, for direct comparison:

Project Stage	24h R ²	Note
Initial models (21 features, 2 years, uncorrected target)	0.271	First working end-to-end pipeline
Expanded features (42 features, 2 years, uncorrected target)	0.299	Below supervisor's required standard
Corrected AQI target (EPA averaging periods), 2 years	0.55	Root-cause fix — single largest single improvement

+ Extended lag/rolling features	0.57	Incremental
+ Genuine future weather forecast features	0.66	Large gain, especially at 48h/72h
+ Four years of history	0.87 (favourable holdout) / 0.63 (rigorous backtest)	Final, production figure

The final, honestly-reported (three-season backtest) accuracy represents more than a doubling of the model's explanatory power (R^2) relative to the initial submission, achieved primarily through correcting the target variable's definition, extending the historical dataset, and incorporating genuine future weather information — rather than through model choice or hyperparameter tuning alone.

20. Limitations & Future Work

- Ground truth is model-derived, not station-measured. Open-Meteo serves CAMS atmospheric reanalysis data rather than a physical Lahore monitoring station reading. This is consistent between training and serving, but should be understood as modelled rather than directly measured air quality.
- Forecast skill degrades with lead time. Weather forecast accuracy itself degrades the further ahead it looks, which is part of why the 72-hour forecast is materially less accurate than the 24-hour forecast — an inherent limitation of the forecasting problem rather than of this system specifically.
- Further historical depth. Extending the backfilled history beyond four years, capturing additional winter smog seasons, would likely continue to improve accuracy, based on the strong effect observed when extending from two to four years.
- Ensembling. Averaging or stacking predictions across the trained models was not pursued in depth and may offer a modest further accuracy gain.
- Resource-constrained scheduling. The automated pipeline is fully capable of continuous hourly/daily operation as designed; scheduling was intentionally reduced during development to remain within the project's free-tier cloud compute allowance, with full-frequency operation restored for final demonstration.

21. Conclusion

This project delivered a complete, end-to-end, serverless machine learning system for forecasting Lahore's Air Quality Index across three time horizons, fulfilling every stated project requirement: automated hourly data collection, a managed feature store, historical backfill, comparison of five models spanning statistical to deep-learning approaches with rigorous baseline comparison, an automated daily retraining and model-registry pipeline,

SHAP-based explainability, a publicly deployed interactive dashboard with hazardous-AQI alerting, and comprehensive automated testing.

Equally important to the final numbers is the engineering process that produced them. A genuine data-reliability failure was independently diagnosed and resolved early in the project. A supervisor's rigorous accuracy review was met not with defensive justification but with a structured root-cause investigation that uncovered and corrected a genuine, non-obvious error in the target variable's scientific definition — more than doubling model accuracy as a direct result. Dozens of infrastructure, dependency, and deployment issues were systematically diagnosed and resolved, each with a documented root cause rather than a superficial patch. The result is a system whose accuracy claims are backed by an honest, multi-season backtest rather than a single flattering split, whose predictions are explainable rather than opaque, and whose deployment verifies and displays its own data provenance in real time rather than merely asserting it.

The live application is available at: <https://aqipredictionbyadan.streamlit.app/>